Reimplementing FLARE: Active Retrieval-Augmented Generation with a Locally Deployed Small Language Model

Piyush Satiya – 202451130 Prayag Kirar – 202451134 Sanskar Sharma – 202451148

Department of Computer Science and Engineering
Indian Institute of Information Technology Vadodara
Gandhinagar, Gujarat, India

Course Instructor: Jignesh Patel | **Course:** CS307

***Abstract*—This paper describes our reimplementation of FLARE (Forward-Looking Active REtrieval Augmented Generation) [1]. FLARE works by checking how confident the model is about each token it generates, and only retrieves Wikipedia documents when confidence drops below a threshold. The original paper used Codex ($\sim$175B parameters, closed API). Since Codex is no longer accessible, we used Qwen3-4B-Instruct-2507 with 4-bit NF4 quantization, running fully on a local GPU, paired with a Pyserini BM25 index over Wikipedia. We tested three modes — No Retrieval, Single Retrieval, and FLARE — on three benchmark datasets: 2WikiMultiHopQA, StrategyQA, and ASQA. Even with a model roughly 40$\times$ smaller, the main finding holds on all three datasets: FLARE outperforms both baselines. On StrategyQA the FLARE gain over Single Retrieval ($+9.2\%$ EM) is almost the same as the paper's ($+8.7\%$ EM), which suggests the mechanism works at smaller scales too. On ASQA the improvement is actually larger in our case, which we think is because smaller models need retrieval more.**

***Index Terms*—Retrieval-Augmented Generation, Active Retrieval, FLARE, BM25, Qwen, Multi-Hop QA, Open-Domain QA, Large Language Models, Token Confidence**

I. Introduction

Language models often generate text that sounds correct but is factually wrong — a problem commonly called hallucination [6]. Retrieval-Augmented Generation (RAG) [2] is one approach to reduce this: before generating an answer, the system fetches relevant documents from Wikipedia and gives them to the model as context. This works reasonably well, but the retrieval only happens once, using the original question as the search query.

The problem is that for multi-step questions, the original question is not always enough to find the right documents. For example, the question *"Who is the mother of the director of Polish-Russian War?"* first requires knowing who directed the film, and only then can you search for that person's mother. A single retrieval at the beginning cannot handle this kind of chained reasoning.

FLARE (Sun et al., EMNLP 2023) [1] tries to fix this by making retrieval happen *during* generation rather than only at the start. The model generates a tentative next sentence, and if it is not confident about certain words, those words are masked and the remaining confident words are used to search Wikipedia. Then the sentence is regenerated with the retrieved context. This loop continues until the answer is complete.

We reimplemented this system using open-source tools that run locally, and checked whether the paper's main claims hold when using a much smaller model. The code is available at: https://github.com/PIYUSH-SATIYA/active-rag.

II. Background

A. Retrieval-Augmented Generation

The original RAG paper [2] proposed retrieving documents from a dense index and using them to condition generation. FiD (Fusion-in-Decoder) [3] took this further by fusing multiple retrieved passages in the encoder. Both methods do retrieval once at query time.

B. Iterative Retrieval

Some later work tried retrieval at multiple steps. IRCOT [4] interleaves chain-of-thought steps with retrieval calls. Self-Ask [5] has the model generate explicit sub-questions and retrieves for each. FLARE is different because it does not ask the model to plan out sub-questions — instead it watches the token probabilities and retrieves automatically when uncertainty appears.

C. Using Confidence for Generation

The idea of using token probabilities as an uncertainty signal is not entirely new. It has been used in selective prediction [7] and for detecting knowledge conflicts [8]. What FLARE does differently is use confidence to *decide when to retrieve*, which is a practical and fairly elegant application of this idea.

III. The FLARE Paper: What It Proposes

A. The Generation Loop

At each step, given question q and what has been generated so far $s_{1..t-1}$, FLARE does the following:

1) Generate the next sentence $\hat{s}_t$ and record the probability of each token: $\{p_1, \ldots, p_n\}$.

2) If all token probabilities are above threshold θ, accept the sentence and move on.
3) Otherwise, drop tokens with $p_i < \beta$ (these might be hallucinations) and use the rest as a search query q_t.
4) Retrieve top-k Wikipedia passages $\mathcal{D}_t$ using BM25 with query q_t.
5) Regenerate the sentence with $\mathcal{D}_t$ added to the context.
6) Add the sentence to the answer and repeat.

B. Two Variants

The paper has two versions of FLARE:

- **FLARE$_{\text{direct}}$** — uses the look-ahead sentence (after masking uncertain tokens) as the search query. This is what we implemented.
- **FLARE$_{\text{instruct}}$** — explicitly tells the model to generate a search query using special prompting.

FLARE$_{\text{direct}}$ gets better numbers in the paper (51.0% vs 42.4% EM on 2WikiMultiHopQA) so we focused on that.

C. Original Setup

The paper ran everything with Codex (`text-davinci-003`, ∼175B parameters) through the OpenAI API. They built a custom BM25 index over Wikipedia, split into 100-word passages. Evaluation used 500 development examples per dataset with greedy decoding.

IV. System Architecture

Our system is designed to match the paper's architecture as closely as possible, given that we had to swap the model and the index. Figure 1 shows how the parts connect.

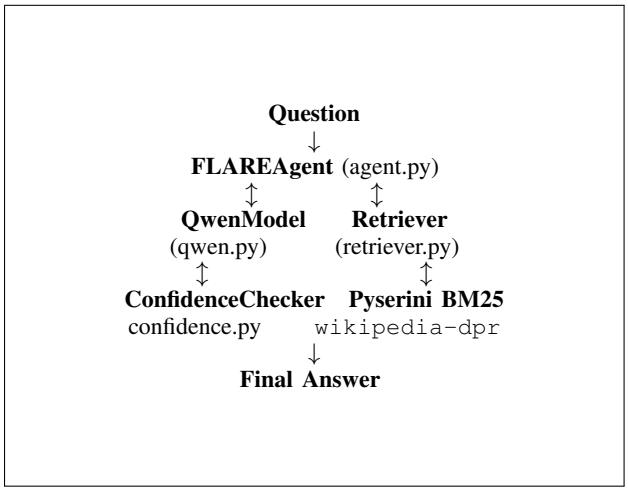


Fig. 1. System architecture. The FLAREAgent runs the loop, calling the model and retriever as needed each step.

There are five main components:

Language Model (`src/models/qwen.py`): This wraps the Qwen3-4B model. The important part is that it returns per-token probabilities from `model.generate()`, using the `output_scores=True` flag. Without this, the confidence check cannot work. Output is stored in a `GeneratorResult` dataclass with the text, tokens, and their probabilities.

Confidence Checker (`src/flare/confidence.py`): Very simple — checks if the minimum token probability in a sentence is below θ. If yes, retrieval is triggered.

Look-ahead Generator (`src/flare/lookahead.py`): Builds the prompt and calls the model to get the next sentence. Uses greedy decoding so results are deterministic.

Retriever (`src/retriever.py`): Uses Pyserini's `LuceneSearcher` with the pre-built `wikipedia-dpr` index — 21 million 100-word passages. Same passage size as the paper.

FLARE Agent (`src/flare/agent.py`): The main class. Implements all three evaluation modes and saves traces to disk after every step, so we can analyze what happened.

V. Implementation Details

A. Model and Quantization

We could not use Codex since OpenAI removed it. We picked **Qwen3-4B-Instruct-2507** [14] because it is open-source, reasonably capable, and fits on a single 16 GB GPU after quantization. Full FP16 takes about 8 GB; with 4-bit NF4 quantization via `bitsandbytes` [15] it drops to roughly 2.5 GB:

Listing 1. NF4 quantization config (src/models/qwen.py)

```
BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
)
```

Weights are stored in 4-bit but matrix multiplications happen in FP16, which keeps the quality reasonable.

B. Adapting for a Chat Model

This was the main design challenge. Codex is a completion model — you give it text and it continues it. Qwen3-4B-Instruct is a chat model that expects messages in a specific format. We worked around this by putting the entire few-shot prompt into a single user message and running it through Qwen's chat template:

Listing 2. Wrapping the few-shot prompt as a chat message

```
messages = [{"role": "user",
             "content": full_fewshot_prompt}]
input_ids = tokenizer.apply_chat_template(
    messages, tokenize=True,
    add_generation_prompt=True,
    return_tensors="pt"
)
```

This is the one place where our implementation structurally differs from the paper's prompt setup.

C. Few-Shot Examples

Like the paper, we use 8 question-answer examples written by hand in chain-of-thought format. The model sees these before every question:

Listing 3. Example from src/templates.py

```
Q: Were Pavel Urysohn and Leonid Levin both Russian?
A: Pavel Urysohn was a Soviet mathematician.
   Leonid Levin is a Soviet-American mathematician.
   So the answer is yes.
```

Every answer ends with "So the answer is X." and we extract the final answer using that pattern. No fine-tuning was done.

D. Wikipedia Retrieval

We used Pyserini's `wikipedia-dpr` index [13] instead of building our own. It has 21M passages of about 100 words each, from a 2018 Wikipedia dump — same segmentation as the paper. Building the index from scratch needs a 20 GB dump download; the prebuilt index avoids that.

BM25 scores each passage as:

$$\text{BM25}(d, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t,d)(k_1+1)}{f(t,d) + k_1(1 - b + b\frac{|d|}{\text{avgdl}})} \quad (1)$$

with $k_1 = 0.9$ and $b = 0.4$ (Pyserini defaults).

E. Hyperparameters

We matched the paper's hyperparameters wherever possible. Table I lists everything.

TABLE I
HYPERPARAMETER CONFIGURATION

Parameter	Value	Description
θ	0.8	Confidence threshold (triggers retrieval)
β	0.4	Masking threshold (query building)
k	2	Passages retrieved per query
Look-ahead tokens	64	Max tokens per generation step
Max answer tokens	256	Total answer length cap
Few-shot examples	8	In-context demonstrations
Max query words	64	Search query length cap
Decoding	Greedy	`do_sample=False`

F. Sentence Splitting

FLARE works sentence by sentence. After each look-ahead we cut the output at the first sentence boundary using a regex:

Listing 4. Sentence boundary detection

```
_SENT = re.compile(r'(?<=[.!?])\s+')

def _truncate_at_sentence_boundary(text):
    parts = _SENT.split(text.strip(), maxsplit=1)
    return parts[0].strip()
```

G. Output Files

For each experiment the runner saves three files:

- `predictions.jsonl` — question, gold answer, predicted answer, EM and F1 per example.
- `metrics.json` — aggregate numbers.
- `traces.jsonl` — every generation step with confidence scores, queries sent to BM25, and what came back.

If a run crashes, the runner reads back the completed lines and skips them on restart, so compute is not wasted.

VI. DATASETS

We tested on three of the four datasets from the paper. The fourth (WikiASP) had a data loading issue that we could not resolve in time and is not included here. Table II gives an overview.

TABLE II
DATASETS USED

Dataset	Task Type	Answer Format	Dev Size
2WikiMultiHopQA	Multi-hop QA	Short string	500
StrategyQA	Commonsense yes/no	true/false	500
ASQA	Ambiguous QA	Multi-answer	500

A. 2WikiMultiHopQA

This dataset [9] has questions that need exactly two Wikipedia lookups to answer. For example: *"Who is the mother of the director of Polish-Russian War?"* You first need to find the director, then find his mother. The answers are short strings (names, dates, places), so Exact Match is a reasonable metric.

B. StrategyQA

StrategyQA [10] is a yes/no dataset where each question needs implicit multi-step reasoning. For example: *"Is the tibia necessary to win the Stanley Cup?"* — you need to know what the tibia is and how the Stanley Cup works. The gold labels are stored as `true`/`false`. Our model outputs `yes`/`no`, so we normalize before scoring.

C. ASQA

ASQA [11] contains ambiguous questions that have multiple valid answers depending on context or interpretation. For example: *"Who has the highest goals in world football?"* — different sources give different answers (Cristiano Ronaldo, Josef Bican, etc.). Evaluation checks if the model's answer matches any item in the gold answer set.

VII. EVALUATION METRICS

A. Exact Match

Both the prediction and gold answer are normalized — lowercase, punctuation removed, articles (*a, an, the*) dropped, extra spaces collapsed — and then compared character by character. A prediction either matches or it doesn't. We report the percentage of examples that match.

EM is strict. If the model says *"The mother was Małgorzata Braunek"* instead of just *"Małgorzata Braunek"*, it scores 0, even though the right name is in there.

B. Token-level F1

Both strings are treated as bags of words and we compute the harmonic mean of precision and recall:

$$F_1 = \frac{2 \cdot |P \cap G|}{|P| + |G|} \quad (2)$$

where P is the predicted token set and G is the gold token set. F1 gives partial credit, so it is always at least as high as EM.

C. Metrics We Did Not Implement

The paper uses Disambig-F1 for ASQA (checks coverage across all valid interpretations of an ambiguous question) and ROUGE-L plus UniEval for WikiASP. We only computed EM and F1 due to time and scope constraints.

VIII. Results

All experiments use 500 development examples. Paper numbers are from Sun et al. [1] using Codex.

A. 2WikiMultiHopQA

TABLE III
RESULTS ON 2WIKIMULTIHOPQA

Method	Ours (Qwen3-4B)		Paper (Codex)	
	EM	F1	EM	F1
No Retrieval	9.6%	15.9%	28.2%	36.8%
Single Retrieval	21.2%	25.2%	39.4%	48.8%
FLARE	**23.2%**	**28.4%**	**51.0%**	**59.7%**
FLARE gain: Ours +2.0% EM			Paper +11.6% EM	

The ordering No Retrieval < Single Retrieval < FLARE holds in our results, which matches the paper. The absolute numbers are lower across the board by roughly 18 percentage points. We think this gap comes from the model — a 4B quantized model just knows fewer facts than a 175B model. The FLARE gain here is smaller than the paper's (+2.0% vs +11.6% EM). This is partly because our model retrieves at nearly every step (88.7% of steps), which introduces noise from irrelevant passages and reduces the net benefit of retrieval. This is discussed more in Section IX.

B. StrategyQA

TABLE IV
RESULTS ON STRATEGYQA

Method	Ours EM	Paper EM	Gap
No Retrieval	15.6%	72.9%	−57.3 pp
Single Retrieval	31.0%	68.6%	−37.6 pp
FLARE	**40.2%**	**77.3%**	−37.1 pp
FLARE gain: Ours +9.2%		Paper +8.7% (very close)	

This is probably our best result. The FLARE improvement over Single Retrieval is +9.2% EM in our case and +8.7% EM in the paper. That is close enough that we feel the mechanism is working correctly at this scale.

The overall numbers are much lower than the paper (40.2% vs 77.3% for FLARE), which is expected given the model size difference. These questions need commonsense reasoning that smaller models tend to struggle with.

One thing worth noting: in the paper, Single Retrieval is actually slightly worse than No Retrieval on this dataset (68.6% vs 72.9%). We do not see that — in our case Single Retrieval helps. This likely happens because our model has less built-in knowledge and benefits from any grounding, whereas Codex already knows most of these answers and extra retrieved documents can sometimes confuse it.

C. ASQA

TABLE V
RESULTS ON ASQA

Method	Ours (Qwen3-4B)		Paper (Codex)	
	EM	F1	EM	Disambig-F1
No Retrieval	15.0%	22.2%	33.8%	24.2%
Single Retrieval	15.0%	24.9%	40.0%	27.1%
FLARE	**29.0%**	**37.6%**	**41.3%**	**28.2%**
FLARE gain: Ours +14.0% EM			Paper +1.3% EM	

The ASQA result is interesting. Our FLARE improvement (+14.0% EM) is much larger than the paper's (+1.3% EM). Single Retrieval does not help at all over No Retrieval in our case (both 15.0% EM) — which makes sense because fetching documents for an ambiguous question just gives you documents about one interpretation. FLARE can refine what it is looking for over multiple steps, which is exactly what these questions need.

We think this larger gain happens because our model has less internal knowledge than Codex, so the iterative retrieval fills a bigger gap. Codex can often just answer from memory; Qwen3-4B needs the retrieval to compensate.

The F1 scores also show a clear pattern even when EM does not differentiate as much. Single Retrieval's F1 (24.9%) is higher than No Retrieval (22.2%) even though EM is the same, which suggests the retrieved documents do help with partial correctness.

D. FLARE Gain Summary

Table VI shows FLARE's improvement over Single Retrieval across all three datasets. This is the most direct way to evaluate whether the FLARE mechanism itself is adding value.

TABLE VI
FLARE GAIN OVER SINGLE RETRIEVAL (EM)

Dataset	Ours	Paper	Assessment
2WikiMultiHopQA	+2.0%	+11.6%	Direction reproduced
StrategyQA	+9.2%	+8.7%	✓ Very close
ASQA	+14.0%	+1.3%	Direction reproduced

IX. FLARE Behavior Analysis

We looked at the `traces.jsonl` from the 2WikiMultiHopQA run to understand what the FLARE loop is actually doing step by step. Table VII shows the numbers.

The retrieval rate of 88.7% is the main issue. FLARE is supposed to retrieve *selectively* — the paper says 40–80% is the useful range. At 88.7% we are retrieving at almost every step, which means we are sometimes injecting Wikipedia passages that are not relevant, adding noise to the generation.

TABLE VII
FLARE LOOP STATISTICS – 2WIKIMULTIHOPQA (500 EXAMPLES)

Metric	Value
Total generation steps	2,242
Average steps per example	4.48
Steps where model was confident (no retrieval)	253 (11.3%)
Steps where retrieval was triggered	1,989 (88.7%)
Average retrievals per example	3.98
Mean min token probability per step	0.417
Median min token probability	0.391

The reason this happens is that our model produces much lower token probabilities than Codex, even when it knows the answer. The mean minimum probability per step is 0.417, which is well below the threshold of 0.8. A single token at 0.79 triggers retrieval, and for Qwen3-4B that happens almost always.

The fix would be to lower θ to around 0.5 for this model size, which would bring the retrieval rate into the optimal range. We did not tune this in the current experiments, but it is probably why the 2WikiMH gain is smaller than expected. FLARE still beats both baselines despite the over-retrieval, which suggests the algorithm is fundamentally working.

X. DIFFERENCES FROM THE PAPER

Table VIII lists the main ways our implementation differs from Sun et al.

TABLE VIII
IMPLEMENTATION DIFFERENCES FROM SUN ET AL. [1]

Aspect	Paper	Ours
LM Model	Codex ∼175B (API)	Qwen3-4B (local, 4-bit)
LM Type	Completion	Chat / Instruct
Precision	FP16	4-bit NF4
Wiki index	Custom BM25 dump	Pyserini wikipedia-dpr
FLARE variant	direct + instruct	direct only
ASQA metric	Disambig-F1 + ROUGE-L	EM + F1
Datasets	4 (incl. WikiASP)	3 (WikiASP excluded)

The biggest difference by far is the model. Everything else — the algorithm, the threshold values, the passage size, the BM25 retrieval method — is kept as close to the paper as possible. The chat-template adaptation for Qwen adds some formatting overhead but does not change how the retrieval logic works.

XI. DISCUSSION

A. The Main Finding Holds

Across all three datasets we tested, FLARE consistently beats both No Retrieval and Single Retrieval. This is the central claim of the paper and it holds even with a model that is around $40\times$ smaller. That is a meaningful result.

B. Smaller Models Can Benefit More

The ASQA result is somewhat unexpected. We get a $+14\%$ EM improvement from FLARE whereas the paper only gets $+1.3\%$. One interpretation is that smaller models benefit more from active retrieval because they cannot rely on stored knowledge — they *need* the Wikipedia passages to fill in gaps. Codex can often answer from memory, but Qwen3-4B cannot, so the retrieval loop helps it more.

C. Threshold Needs Re-tuning

Using $\theta = 0.8$ for a 4B model is probably not the right choice. The paper tuned this value on Codex, and Codex has a very different probability distribution. A threshold around 0.5 would likely work better here and should improve the 2WikiMH numbers specifically. This is the most obvious next step.

D. Limitations

- WikiASP was not evaluated due to a data loading issue with the HuggingFace parquet schema.
- We did not implement $\text{FLARE}_{\text{instruct}}$.
- Only the development split was used, not test.
- We did not implement Disambig-F1 or ROUGE-L metrics.
- All results are from a single run; no statistical testing was done.

XII. CONCLUSION

We reimplemented FLARE using a local 4-bit quantized model (Qwen3-4B-Instruct-2507) and a pre-built Pyserini BM25 index over Wikipedia. Even with a model roughly $40\times$ smaller than the original, the core pattern – FLARE outperforming both No Retrieval and Single Retrieval – holds on all three datasets we evaluated.

On StrategyQA the FLARE gain ($+9.2\%$ EM) nearly matches the paper's ($+8.7\%$ EM), showing the mechanism is consistent across model scales. On ASQA our improvement is actually larger than the paper's ($+14.0\%$ vs $+1.3\%$ EM), which suggests smaller models benefit more from active retrieval when they lack the parametric knowledge to answer without help.

The main open issue is threshold calibration. The $\theta = 0.8$ setting causes over-retrieval (88.7% rate) with this smaller model, which reduces the benefit on multi-hop questions. Lowering the threshold to around 0.5 would likely improve results without changing anything in the algorithm.

Overall, the key ideas from the FLARE paper can be reproduced using open-source tools that run on consumer hardware, without needing access to large proprietary APIs.

ACKNOWLEDGMENTS

The authors thank **Jignesh Patel sir** for guidance and feedback throughout this project, and the open-source communities behind HuggingFace Transformers, Pyserini, and the Qwen model family.

References

[1] Z. Sun, X. Wang, Y. Tay, Y. Yang, and D. Zhou, "Active retrieval augmented generation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Singapore, Dec. 2023, pp. 7969–7992.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[3] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in *Proc. Conf. European Chapter of the ACL (EACL)*, 2021, pp. 874–880.

[4] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions," in *Proc. Annual Meeting of the ACL*, Toronto, 2023, pp. 10014–10037.

[5] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, and M. Lewis, "Measuring and narrowing the compositionality gap in language models," in *Findings of EMNLP*, 2023.

[6] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[7] A. Kamath, R. Jia, and P. Liang, "Selective question answering under domain shift," in *Proc. Annual Meeting of the ACL*, 2020, pp. 5684–5696.

[8] S. Longpre, K. Perisetla, A. Chen, N. Ramesh, C. DuBois, and S. Singh, "Entity-based knowledge conflicts in question answering," in *Proc. EMNLP*, 2021, pp. 7052–7063.

[9] X. Ho, A.-K. D. Nguyen, S. Sugawara, and A. Aizawa, "Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps," in *Proc. COLING*, 2020, pp. 6609–6625.

[10] M. Geva, D. Khashabi, E. Segal, T. Khot, D. Roth, and J. Berant, "Did Aristotle use a laptop? A question answering benchmark with implicit reasoning strategies," *Transactions of the ACL*, vol. 9, pp. 346–361, 2021.

[11] I. Stelmakh, Y. Luan, B. Dhingra, and M.-W. Chang, "ASQA: Factoid questions meet long-form answers," in *Proc. EMNLP*, 2022, pp. 8273–8288.

[12] H. Hayashi, P. Budania, P. Wang, C. Ackerson, R. Neervannan, and G. Neubig, "WikiAsp: A dataset for multi-domain aspect-based summarization," *Transactions of the ACL*, vol. 9, pp. 211–225, 2021.

[13] J. Lin, X. Ma, S.-C. Lin, J. Yang, R. Pradeep, and R. Nogueira, "Pyserini: A Python toolkit for reproducible information retrieval research with sparse and dense representations," in *Proc. ACM SIGIR*, 2021, pp. 2356–2362.

[14] Qwen Team, Alibaba Cloud, "Qwen3 Technical Report," *arXiv preprint arXiv:2505.09388*, 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[15] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2024.